Київський національний університет ім. Тараса Шевченка

Кафедра мережевих та інтернет технологій

Лабораторна робота № 2

Дисципліна: Системи програмування

Тема: Паралелізм даних та паралелізм задач (ч.1)

Виконав: Студент групи МІТ-41

Пугач Назар

Хід роботи

Завдання 1: Реалізуйте програму на C#, яка створює дві задачі (Task) для паралельного виконання:

- перша задача виводить числа від 1 до 10 з паузою 200 мс;
- друга задача виводить літери від A до J з паузою 200 мс.

Використайте методи Start() та WaitAll().

```
using System;
using System.Threading;
using System.Threading.Tasks;

class Program
{
    static void Main()
    {
        Task task1 = new Task(() =>
        {
            for (int i = 1; i <= 10; i++)
            {
                Console.WriteLine($"Number: {i}");
                Thread.Sleep(200);
            }
        });

        Task task2 = new Task(() =>
        {
            for (char c = 'A'; c <= 'J'; c++)
            {
                Console.WriteLine($"Letter: {c}");
                Thread.Sleep(200);
            }
        });

        task1.Start();
        task2.Start();

        Task.WaitAll(task1, task2);

        Console.WriteLine("Both tasks have completed.");
    }
}
```

Рисунок 2.1 – Код програми

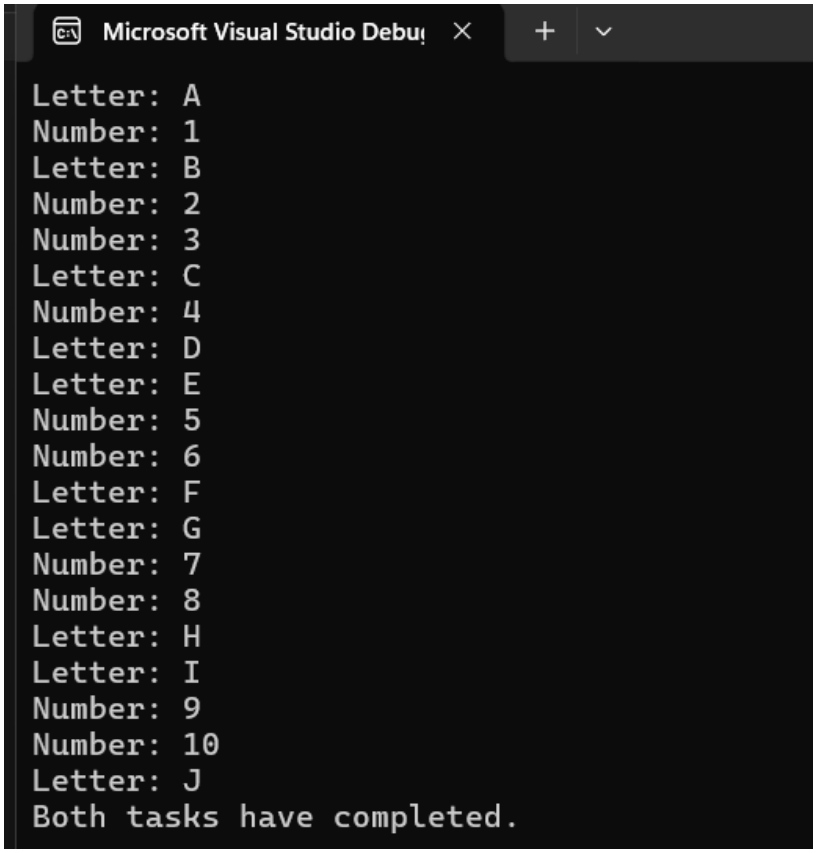

Рисунок 2.2 – Результат виконання роботи

Питання для звіту: поясніть різницю між класами Thread та Task.

Thread – це низькорівневий клас для створення та керування потоками виконання. Він створює окремий потік операційної системи і дає повний контроль над його параметрами (наприклад, пріоритетом або станом), але потребує більше ресурсів і складніший у використанні.

Task – це високорівнева абстракція для виконання асинхронних або паралельних операцій. Зазвичай Task використовує ThreadPool, що дозволяє ефективніше керувати потоками та спрощує написання паралельного коду.

Завдання 2: Створіть програму, яка запускає три задачі, кожна з яких виконує підрахунок до 5. Виведіть у консоль ідентифікатор (Id) кожної задачі та поточний ідентифікатор (CurrentId) під час виконання.

```
using System;
using System.Threading;
using System.Threading.Tasks;

class Program
{
    static void Main()
```

```
    {
        Task t1 = Task.Run(() => Count());
        Task t2 = Task.Run(() => Count());
        Task t3 = Task.Run(() => Count());

        Task.WaitAll(t1, t2, t3);
    }

    static void Count()
    {
        for (int i = 1; i <= 5; i++)
        {
            Console.WriteLine($"Task Id: {Task.CurrentId}, Iteration:
{i}");
            Thread.Sleep(200);
        }
    }
}
```

Рисунок 2.3 – Код програми

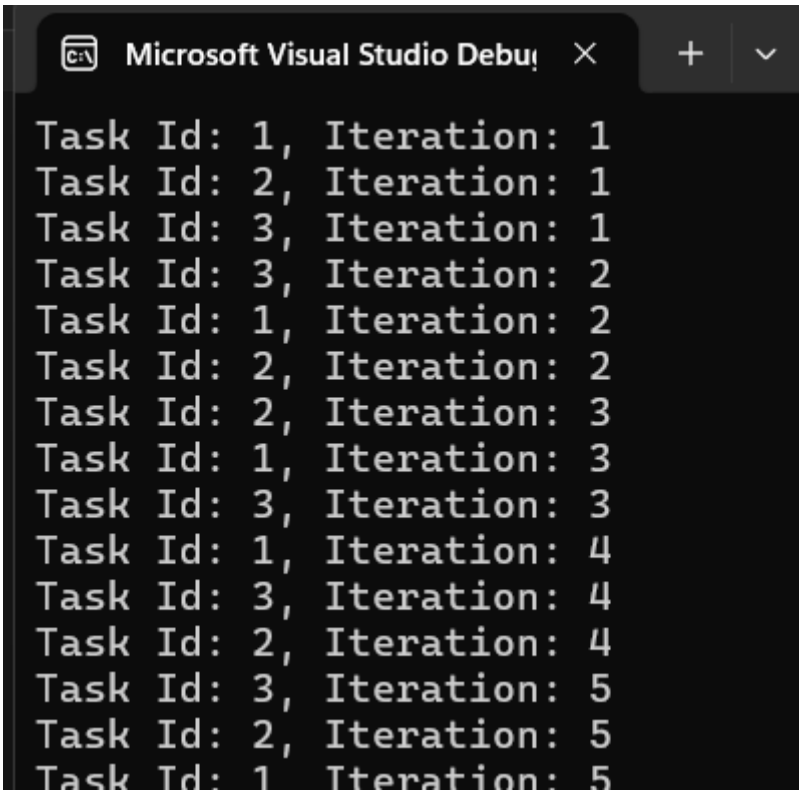

Рисунок 2.4 – Результат роботи програми

Питання для звіту: чи є ідентифікатори задач впорядкованими? Як їх можна використати для відстеження виконання?

Ідентифікатори задач не гарантують впорядкованого виконання. Вони зазвичай призначаються послідовно під час створення задач, але самі задачі виконуються паралельно, тому порядок їх виконання та виведення в консоль може бути різним.

Ідентифікатори задач можна використовувати для розрізнення задач у паралельному виконанні, логуванні виконання задач, відстеження стану задач під час дебагу, аналізу порядку виконання в багатопоточних програмах. Наприклад, у логах можна бачити, яка саме задача виконала певну операцію.

Завдання 3: Реалізуйте програму, яка:

- створює задачу для обчислення суми чисел від 1 до N (N вводиться користувачем).
- використовує метод ContinueWith() для запуску другої задачі, яка виводить результат у консоль.
- використовує лямбда-вираз для реалізації продовження.

```
using System;
using System.Threading.Tasks;

class Program
{
    static void Main()
    {
        Console.Write("Input N: ");
        int N = int.Parse(Console.ReadLine());

        Task<int> sumTask = Task.Run(() =>
        {
            int sum = 0;
            for (int i = 1; i <= N; i++)
                sum += i;
            return sum;
        });

        sumTask.ContinueWith(t =>
        {
            Console.WriteLine($"The sum from 1 to {N} = {t.Result}");
        });

        Console.ReadLine();
    }
}
```

Рисунок 2.5 – Код програми

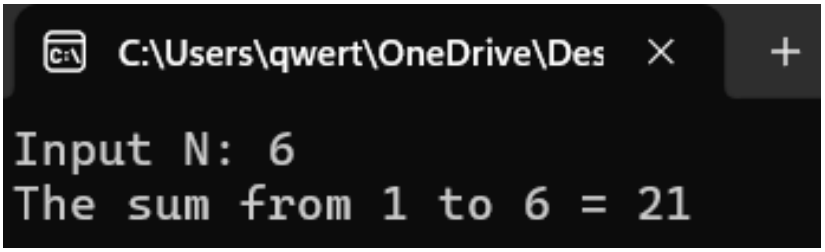

Рисунок 2.6 – Результат роботи коду

Питання для звіту: які переваги має використання продовжень задач у порівнянні з класичним підходом синхронізації?

Використання продовжень задач (ContinueWith) дозволяє організувати неблокуюче виконання, оскільки наступна задача запускається автоматично після завершення попередньої без зупинки основного потоку. Код стає більш зрозумілим і структурованим. Задачі використовують ThreadPool, що забезпечує ефективніше використання системних ресурсів. Також це спрощує створення складних асинхронних сценаріїв, де одна операція залежить від результату іншої.

Завдання 4: Розробіть програму, яка виконує три методи паралельно за допомогою Parallel.Invoke():

- метод 1: обчислює факторіал числа.
- метод 2: обчислює суму чисел від 1 до N.
- метод 3: виводить повідомлення у консоль з паузою 300 мс.

```
using System;
using System.Threading;
using System.Threading.Tasks;

class Program
{
    static void Main()
    {
        Console.Write("Enter a number for factorial and sum calculation: ");
        int N = int.Parse(Console.ReadLine());

        Parallel.Invoke(
            () => Factorial(N),
            () => Sum(N),
            () => PrintMessage()
        );

        Console.WriteLine("All methods have completed.");
    }

    static void Factorial(int n)
    {
        long fact = 1;
        for (int i = 1; i <= n; i++)
            fact *= i;

        Console.WriteLine($"Factorial of {n} = {fact}");
    }

    static void Sum(int n)
    {
        int sum = 0;
        for (int i = 1; i <= n; i++)
            sum += i;

        Console.WriteLine($"Sum from 1 to {n} = {sum}");
    }

    static void PrintMessage()
    {
        for (int i = 0; i < 5; i++)
```

```
            {
                Console.WriteLine("Method 3 is running...");
                Thread.Sleep(300);
            }
        }
    }
```

Рисунок 2.7 – Код програми

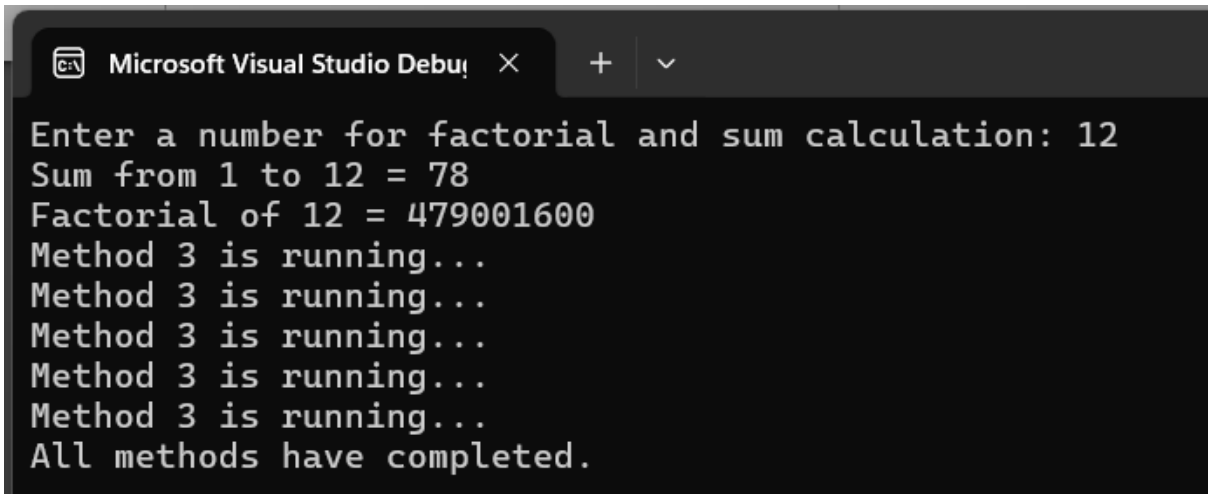

Рисунок 2.8 – Результат роботи коду

Питання для звіту: які переваги має використання Parallel.Invoke() у порівнянні зі створенням окремих задач?

Використання Parallel.Invoke() має кілька переваг у порівнянні зі створенням окремих задач. Цей метод дозволяє простим способом запускати кілька незалежних методів паралельно, без необхідності вручну створювати, запускати та синхронізувати задачі. Parallel.Invoke() автоматично використовує ThreadPool для ефективного керування потоками та самостійно очікує завершення всіх переданих методів. Це зменшує кількість коду, спрощує програму та підвищує продуктивність, особливо коли потрібно паралельно виконати кілька незалежних операцій.

Висновок: Під час виконання лабораторної роботи я навчився з паралельними задачами.